

EXPERIMENT NO - 3

AIM

To demonstrate Interprocess Communication (IPC) using Shared Memory and Message Queue.

SOFTWARE REQUIRED

A Linux operating system

GCC (GNU Compiler Collection) for compiling C programs

Terminal or command line interface

THEORY

Interprocess Communication (IPC) is a mechanism that allows processes to communicate with each other and synchronize their actions. The communication can be seen as a method of cooperation between them.

- **Shared Memory:** An efficient means of passing data between programs. One process will create a memory portion which other processes (if permitted) can access. System calls used include `shmget()`, `shmat()`, and `shmdt()`.
- **Message Queue:** A linked list of messages stored within the kernel and identified by a message queue identifier. A process can write a message to the queue using `msgsnd()`, and another process can read it using `msgrcv()`.

PROCEDURE

1. Write a C program to implement Shared Memory.
2. Use `ftok()` to generate a unique key, and `shmget()` to allocate a shared memory segment.
3. Attach the memory segment using `shmat()`, write data into it, and detach using `shmdt()`.
4. Write another C program (or use a child process) to attach to the same memory segment and read the data.

Repeat a similar structure using `msgget()`, `msgsnd()`, and `msgrcv()` to demonstrate Message Queue communication.

INPUT-

sender.c

```
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/msg.h>

#define MAX 100

struct mesg_buffer {
    long mesg_type;
    char mesg_text[MAX];
} message;

int main() {
    key_t key;
    int msgid;

    // ftok to generate unique key
    key = ftok("progfile", 65);

    // msgget creates a message queue and returns identifier
    msgid = msgget(key, 0666 | IPC_CREAT);
    message.mesg_type = 1;

    printf("Write Data : ");
    fgets(message.mesg_text, MAX, stdin);

    // msgsnd to send message
    msgsnd(msgid, &message, sizeof(message), 0);
    printf("Data send is : %s \n", message.mesg_text);

    return 0;
}
```

RECIVER.C

```
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/msg.h>

#define MAX 100

struct mesg_buffer {
```

```

    long mesg_type;
    char mesg_text[MAX];
} message;

int main() {
    key_t key;
    int msgid;

    // ftok to generate unique key
    key = ftok("progfile", 65);

    // msgget creates a message queue and returns identifier
    msgid = msgget(key, 0666 | IPC_CREAT);

    // msgrcv to receive message
    msgrcv(msgid, &message, sizeof(message), 1, 0);

    printf("Data Received is : %s \n", message.mesg_text);

    // to destroy the message queue
    msgctl(msgid, IPC_RMID, NULL);

    return 0;
}

```

OUTPUT

Stop copy pasting code you don't actually understand

Build the coding confidence you need to become a developer companies will fight for

Become a PRO

Programiz

C Online Compiler

GADU- prefabricated, containerized, modular data center solution

Deploy your AI factory instantly, GADU - GIGABYTE Accelerated Deployment Unit, GADU delivers an entire high-density data center in a standardized, transportable container...

LEARN MORE

SPONSORED BY GIGABYTE

Programiz PRO >

main.c

Run

```

1 #include <stdio.h>
2 #include <sys/ipc.h>
3 #include <sys/msg.h>
4
5 #define MAX 100
6
7 struct mesg_buffer {
8     long mesg_type;
9     char mesg_text[MAX];
10 } message;
11
12 int main() {
13     key_t key;
14     int msgid;
15
16     // ftok to generate unique key
17     key = ftok("progfile", 65);
18
19     // msgget creates a message queue and returns identifier
20     msgid = msgget(key, 0666 | IPC_CREAT);
21
22     // msgrcv to receive message
23     msgrcv(msgid, &message, sizeof(message), 1, 0);
24
25     printf("Data Received is : %s \n", message.mesg_text);
26
27     // to destroy the message queue
28     msgctl(msgid, IPC_RMID, NULL);
29
30     return 0;
31 }

```

Output

Data Received is : 12

*** Code Execution Successful ***

Visa Consulting & Analytics...

VISA

VCA Helps Improve Fraud Prevention by 58%

Case study

How a Leading Malaysian Bank Reduced Fraud by 58%

As fraud threats evolve, financial institutions need more than detection tools. They need the ability to continuously optimize performance. Visa Consulting & Analytics partnered with a leading...

Download document

Stop copy pasting code you don't actually understand
Build the coding confidence you need to become a developer companies will fight for

Become a PRO

Programiz
C Online Compiler

Programiz PRO

main.c

Run

```
1 #include <stdio.h>
2 #include <sys/ipc.h>
3 #include <sys/msg.h>
4
5 #define MAX 100
6
7 struct msg_buffer {
8     long msg_type;
9     char msg_text[MAX];
10 } message;
11
12 int main() {
13     key_t key;
14     int msgid;
15
16     // fork to generate unique key
17     key = ftok("progfile", 65);
18
19     // msgget creates a message queue and returns identifier
20     msgid = msgget(key, 0666 | IPC_CREAT);
21     message.msg_type = 1;
22
23     printf("Write Data : ");
24     fgets(message.msg_text, MAX, stdin);
25
26     // msgsnd to send message
27     msgsnd(msgid, &message, sizeof(message), 0);
28     printf("Data send is : %s\n", message.msg_text);
29
30     return 0;
31 }
```

Output

Write Data : 12
Data send is : 12

*** Code Execution Successful ***

Ad

HP Victus 15-fa2316TX
Intel Core i5...

₹1,08,990 ₹1,35,358 Buy HP Victus 15-fa2316TX Intel Core i5 14th Gen Gaming Laptop (24GB, 512GB SSD...)

Croma

Visit Site

CONCLUSION

This experiment successfully demonstrated Interprocess Communication in Linux. By utilizing message queues, we provided a structured way to pass independent messages between two isolated programs, illustrating robust interprocess communication mechanisms supported by the system.